```
import cv2
import numpy as np
import matplotlib.pyplot as plt
from google.colab import files

# Upload one image
uploaded = files.upload()

# Get image name
image_name = list(uploaded.keys())[0]

# Read image in grayscale
img = cv2.imread(image_name, 0)

# Create a noisy version of the image
noise = np.random.normal(0, 30, img.shape)

noisy_img = img.astype(np.float32) + noise
noisy_img = np.clip(noisy_img, 0, 255).astype(np.uint8)

# Preprocess the noisy image using Gaussian Blur
preprocessed_img = cv2.GaussianBlur(noisy_img, (5, 5), 0)

# Create ORB detector
orb = cv2.ORB_create(nfeatures=500)

# Detect features and descriptors
kp1, des1 = orb.detectAndCompute(img, None)
kp2, des2 = orb.detectAndCompute(noisy_img, None)
kp3, des3 = orb.detectAndCompute(preprocessed_img, None)

# Create Brute Force Matcher
bf = cv2.BFMatcher(cv2.NORM_HAMMING, crossCheck=True)

# Match original image with noisy image
matches_noisy = bf.match(des1, des2)
matches_noisy = sorted(matches_noisy, key=lambda x: x.distance)

# Match original image with preprocessed image
matches_preprocessed = bf.match(des1, des3)
matches_preprocessed = sorted(matches_preprocessed, key=lambda x: x.distance)

# Draw best matches
result_noisy = cv2.drawMatches(
    img, kp1,
    noisy_img, kp2,
    matches_noisy[:30],
    None,
    flags=2
)

result_preprocessed = cv2.drawMatches(
    img, kp1,
    preprocessed_img, kp3,
    matches_preprocessed[:30],
    None,
    flags=2
)

# Display results
plt.figure(figsize=(15, 10))

plt.subplot(2, 2, 1)
plt.imshow(img, cmap="gray")
plt.title("Original Image")
plt.axis("off")

plt.subplot(2, 2, 2)
plt.imshow(noisy_img, cmap="gray")
plt.title("Noisy Image")
plt.axis("off")

plt.subplot(2, 2, 3)
plt.imshow(result_noisy)
plt.title("Matching with Noise")
plt.axis("off")

plt.subplot(2, 2, 4)
plt.imshow(result_preprocessed)
plt.title("Matching after Preprocessing")
plt.axis("off")
```

```
plt.tight_layout()
plt.show()

# Display number of matches
print("Number of matches with noisy image:", len(matches_noisy))
print("Number of matches after preprocessing:", len(matches_preprocessed))
```

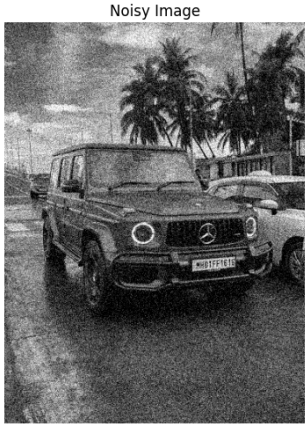
Choose files images (5).jpg

images (5).jpg(image/jpeg) - 40206 bytes, last modified: 07/08/2026 - 100% done

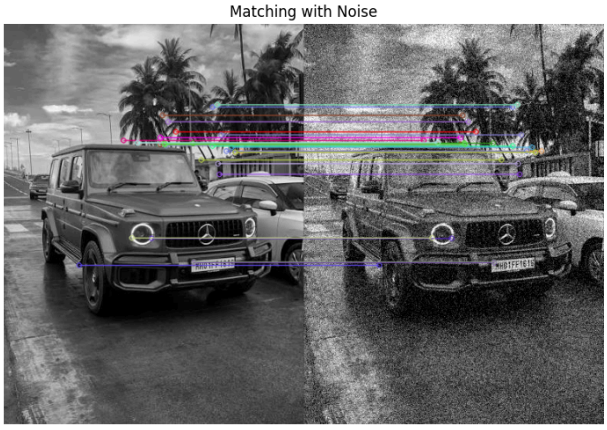
```
Saving images (5).jpg to images (5).jpg
```

Original Image

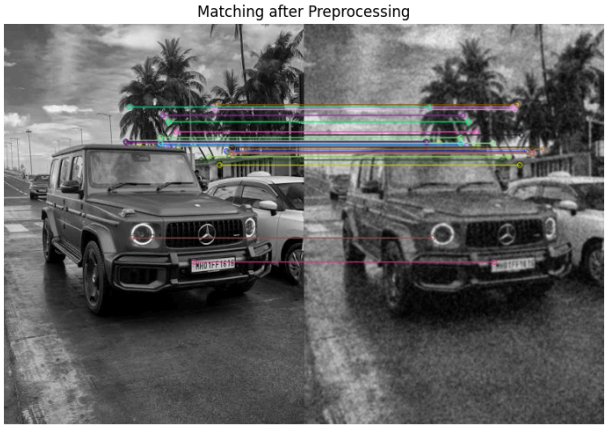
Noisy Image



Matching with Noise



Matching after Preprocessing



```
Number of matches with noisy image: 332
Number of matches after preprocessing: 318
```